DEPLOYMENT GUIDE

Bite Size Theology

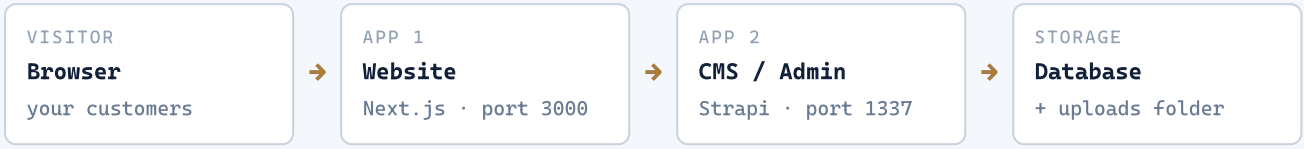
How to put the website and its content management system live on your own hosting — what the system is made of, what you need to have ready, three hosting options, and how to bring the existing content across.

`Next.js 16.2.10` `Strapi 5.50.2` `Node.js 20+` `PostgreSQL / MySQL / SQLite` `PayPal + Stripe`

Prepared for the owner of Bite Size Theology · This document describes the codebase as delivered — every command and setting in it was checked against the actual project files.

01 How the system is built

This is a custom application, not a WordPress-style site. Deploying it means running **two programs** that talk to each other, plus a database and a folder of uploaded images.



The visitor only ever touches the website. The website asks the CMS for text, images, products and prices. You only ever touch the CMS.

The four parts

PART	PORT	WHAT IT DOES
Website Next.js	`3000`	The public site your visitors see — home, podcast, shop, checkout, donation and contact forms.
CMS / Admin Strapi	`1337`	Where you log in to edit every piece of text, upload images, manage products and read store orders. Also serves that content to the website over an API.
Database	internal	Holds all the content records and store orders. Can be PostgreSQL, MySQL or SQLite — the project supports all three.
Uploads folder	—	`cms/public/uploads/` on disk. **Every image you upload is a real file here**, not a row in the database. It must be backed up and moved separately.

Built to fail safely

The website has a complete copy of the original text built into it. If the CMS is switched off, unreachable, or not configured yet, **the site still loads** — it just shows that original text instead of your edits. Likewise, if the payment keys are missing the checkout buttons simply don't appear rather than erroring. You can therefore put the website up first and connect the CMS afterwards.

Who does what

TASK	WHO
First deployment (this document)	A developer, once. Budget half a day.
Editing text, images, prices, products	You, in the CMS admin. No developer, no redeploy.
Reading and fulfilling store orders	You, in the CMS admin under **Order**.
Changing how a page is laid out or built	A developer — that is code.

02 Before you start

Gather these first. Missing one of them mid-deploy is the most common reason a straightforward job turns into a long afternoon.

Hosting

- [ ] **A server with at least 2 GB of RAM.** This is not optional. Building the CMS admin panel runs out of memory on a 1 GB machine and can lock the server up hard enough to need a reboot. 2 GB is the tested minimum.
- [ ] **Command-line (SSH) access** to that server, or a hosting panel that can run Node.js applications.
- [ ] **Node.js 20 or newer** if you are not using Docker. The CMS declares support for Node 20 through 26.

Domain

- [ ] Your main domain, e.g. `bitesizetheology.com` — for the website.
- [ ] A subdomain for the CMS, e.g. `cms.bitesizetheology.com`. Both should point at your server, and both need an HTTPS certificate.

Accounts and keys

SERVICE	NEEDED?	WHAT FOR
PayPal developer.paypal.com	For selling	A **live** Client ID and Secret. Without them the PayPal button is hidden.
Stripe dashboard.stripe.com	For cards	A live Secret Key (`sk_live_…`). Without it the "Pay with card" option is hidden.
Resend resend.com	For forms	An API key plus a verified sending address, so contact / booking / prayer form submissions reach an inbox. Without it those forms fail to submit.
The codebase	Yes	The project files, plus a content export archive — see the next section.

You can go live without the payment keys

The site is designed so that missing keys hide the relevant feature rather than break the page. It is perfectly reasonable to launch the site first and switch the shop on once your PayPal and Stripe accounts are approved.

03 Your content is not in the code

Read this section before you deploy anything. It is the one step that is easy to miss and expensive to discover late.

The most important thing in this document

The project files contain the **software**. They do not contain your **content**. The database and every uploaded image are deliberately excluded from the code — they are far too large, and they change constantly.

If you deploy the code on its own, the CMS will start up with the original placeholder content and none of your images. Nothing is lost — but nothing you edited has arrived either.

What has to travel separately

ITEM	IN THE CODE?	WHERE IT LIVES
All page text, products, prices, orders	No	The database file / server
Every uploaded image and file	No	`cms/public/uploads/`
Passwords, API keys, secrets	No	Environment settings — Section 7
The website and CMS software	Yes	The project files

Moving the content across

The CMS has a built-in export/import tool that packages the database records *and* the uploaded files into a single archive. Use it — do not try to copy the database file by hand.

STEP 1 — ON THE CURRENT MACHINE, PRODUCE THE ARCHIVE

Stop the CMS first

Shut the CMS down before exporting. Reading and writing that database while it is running can corrupt it.

```
# from the project's cms/ folder, with the CMS stopped
cd cms
npm run strapi export -- --no-encrypt --file bst-content

# produces: bst-content.tar.gz  (content records + uploaded files)
```

Send that archive to whoever is deploying, alongside the code. Treat it as confidential — it contains your store orders and customer details.

STEP 2 — ON THE NEW SERVER, LOAD IT IN

Do this **after** the CMS is running for the first time, and **before** anybody starts editing on the new server.

```
# Docker (Option A):
docker compose cp bst-content.tar.gz cms:/app/bst-content.tar.gz
docker compose exec cms npm run strapi import -- --force --file bst-content.tar.gz

# Plain server (Options B and C):
cd cms
npm run strapi import -- --force --file /path/to/bst-content.tar.gz
```

Import replaces everything

`--force` answers "yes" to the tool's safety prompt. The import **wipes the destination's existing content** and replaces it with the archive. That is exactly what you want on a fresh install, and exactly what you do not want once the site is live and people have been editing. Import once, at the start.

Why the CMS does not overwrite your work

On its very first start the CMS fills in the original text so the site is never blank. It checks each item before writing and skips anything that already exists — so restarting the CMS, or importing your content over the top, will never clobber your edits.

04 Option A — One server with Docker RECOMMENDED

Everything needed for this route is already built into the project. One command starts the database, the CMS and the website together, correctly wired to each other. This is the fastest and the most reliable of the three options.

Suitable hosts: any Linux virtual server with 2 GB+ RAM — Hostinger VPS, AWS Lightsail, DigitalOcean, Hetzner, Linode. Roughly $10–15/month.

1 Prepare the server

ON THE SERVER

Install Docker, then allow your own user to run it:

```
sudo apt-get update && sudo apt-get install -y docker.io docker-compose-plugin
sudo usermod -aG docker $USER
# log out and back in so the group takes effect
```

Some providers offer a "Docker" server image with this already done.

2 Copy the project onto the server

ON THE SERVER

Clone it from your Git repository, or upload the folder over SFTP. Everything from here runs in the project's top-level folder.

3 Create the settings file

ON THE SERVER

```
cp .env.docker.example .env
```

Now open `.env` and fill it in. Generate each secret separately — never reuse one value for two settings, and never use the example placeholders:

```
openssl rand -base64 16     # run once per secret
```

Full explanation of every line is in **Section 7**. The two that matter most here:

- `DATABASE_PASSWORD` — a strong password you invent.
- `STRAPI_PUBLIC_URL` — the address a *visitor's browser* uses to reach the CMS. Use `http://YOUR-SERVER-IP:1337` for a first test, or `https://cms.yourdomain.com` once your domain is set up in step 6.

4 Start everything

ON THE SERVER

```
docker compose up -d --build
```

The first run takes several minutes — it is compiling both applications. When it finishes:

- Website → `http://YOUR-SERVER-IP:3000`
- Admin → `http://YOUR-SERVER-IP:1337/admin`

5 Set up the admin and load your content

IN THE BROWSER

1. Open the admin address and create your administrator account. This is the login you will use from now on.
2. Import your content archive — the commands are in **Section 3**.
3. Create an API token so the website can save store orders — see **Section 8**.

6 Put your domain in front, with HTTPS

ON THE SERVER

Running on a bare IP address and plain `http://` is fine for testing but not for a live shop — card payments and login sessions need HTTPS. Install a reverse proxy (**Caddy** is the least work; it obtains and renews certificates automatically) and route:

- `yourdomain.com` → the website container, port 3000
- `cms.yourdomain.com` → the CMS container, port 1337

Point both DNS records at the server, then remove the public `ports:` entries from `docker-compose.yml` so the containers are only reachable through the proxy.

7 Rebuild after the domain change

ON THE SERVER

Update `STRAPI_PUBLIC_URL` in `.env` to the `https://cms…` address, then:

```
docker compose up -d --build web
```

This step is not optional — see the warning below.

Two settings are frozen into the build

`STRAPI_PUBLIC_URL` and `NEXT_PUBLIC_PAYPAL_CLIENT_ID` are compiled into the website when it is built, not read while it runs. If you change either one you **must** rebuild the website container, or the change will appear to have no effect — typically showing up as broken images or a missing PayPal button.

Everyday commands

COMMAND	WHAT IT DOES
`docker compose ps`	Shows which services are running
`docker compose logs -f cms`	Live log for the CMS (or `web`, `db`)

COMMAND	WHAT IT DOES
`docker compose restart web`	Restarts one service
`docker compose down`	Stops everything. Your data is kept.
`docker compose down -v`	**Deletes the database and all uploaded images.** Never run this on a live site.

05 Option B — Shared Node.js hosting HOSTINGER HPANEL

Running both applications as hPanel "Node.js applications" on a Business or Premium plan, backed by the plan's MySQL database. Cheapest option, and the most fragile.

Know the risk before choosing this route

The CMS is a memory-hungry program that runs continuously. Shared hosting can run it, but if it is slow or keeps stopping unexpectedly, the fix is to move the CMS to a small virtual server and keep the website where it is. Also confirm your plan genuinely includes SSH access — you cannot complete this route without it.

This route needs two code changes first

Unlike Option A, the project is **not** ready for this out of the box. A developer must make these two changes and commit them before you begin:

1. Install the MySQL driver — the project ships with PostgreSQL and SQLite drivers only:

```
cd cms && npm install mysql2
```

2. Create `cms/server.js`, the start file the hosting panel looks for:

```
const { createStrapi } = require("@strapi/strapi");
createStrapi().start();
```

1 Create the database and subdomain HPANEL

Databases → MySQL Databases — create a database and a user, and write down the name, username and password. Then **Domains → Subdomains** — create `cms`. Finally **Advanced → SSH Access** — make sure it is switched on.

2 Upload and install OVER SSH

Keep the two applications in separate folders — the hosting panel maps one folder per app.

```
# get the code onto the server, then:
cp -r site/cms cms          # the CMS lives in the repo's cms/ subfolder
cd ~/site && npm ci --include=dev
cd ~/cms  && npm ci
```

Note the flag

The website must be installed with `--include=dev`. Its styling tools are development dependencies, but they are required to *build* the site. A plain production install fails with "Cannot find module '@tailwindcss/postcss'".

3 Build both applications

OVER SSH

```
cd ~/cms  && NODE_ENV=production npm run build
cd ~/site && npm run build
# the website's compiled output needs these two folders beside it:
cp -r .next/static .next/standalone/.next/static
cp -r public       .next/standalone/public
```

If the build says "Killed"

That is the server running out of memory. Retry with `NODE_OPTIONS=--max-old-space-size=1536 npm run build`, or build on your own computer using the same Node version and upload the finished folders instead.

4 Create the two Node.js applications

HPANEL

Advanced → Node.js → Create application, twice:

	CMS APP	WEBSITE APP
Node version	20 or 22	20 or 22 (match what you built with)
Application root	`cms`	`site/.next/standalone`
Application URL	the `cms.` subdomain	the main domain
Startup file	`server.js`	`server.js`

Do not set `PORT` or `HOST` yourself — the hosting panel assigns those.

5 Enter the settings, deploy CMS first

HPANEL

Each application has an **Environment variables** panel. Fill them from **Section 7**. For the CMS also set:

```
DATABASE_CLIENT=mysql
DATABASE_HOST=localhost
DATABASE_PORT=3306
DATABASE_NAME=your_db_name
DATABASE_USERNAME=your_db_user
DATABASE_PASSWORD=your_db_password
```

Start the CMS before you build the website — the website freezes the CMS address into its build, so that address has to exist first. Enable free SSL (**hPanel → SSL**) on both the domain and the `cms.` subdomain before you finish.

06 Option C — Vercel + a separate CMS host SPLIT

The website goes on Vercel, which builds and hosts Next.js sites with no server administration at all. The CMS still needs a real server of its own.

WHAT THIS IS GOOD AT

- The website side becomes effortless — push a change, it deploys itself.
- Fast worldwide, scales automatically, HTTPS handled for you.
- A generous free tier for the website.

WHAT IT DOES NOT SOLVE

- The CMS *cannot* run on Vercel — it needs continuous running and a writable uploads folder.
- You still need a small server (or Strapi Cloud) for it.
- Two providers, two bills, two things to keep an eye on.

How it goes together

1. **Deploy the CMS first**, on a small virtual server or Strapi Cloud. In practice this is Option A with only the `db` and `cms` services, or Option B's CMS half. Give it its own domain, e.g. `cms.yourdomain.com`, with HTTPS.
2. **Import your content archive** into it — Section 3.
3. **Connect the repository to Vercel.** It detects Next.js and needs no build configuration.
4. **Add the website settings** in Vercel's project settings — the "Website" table in Section 7. `STRAPI_URL` and `STRAPI_PUBLIC_URL` both point at your CMS domain.
5. **Point your main domain** at Vercel and deploy.

The CMS must be reachable from the public internet

Vercel builds and runs your website in its own data centres, so it can only fetch content from a CMS that is publicly available over HTTPS. A CMS locked to a private network or sitting behind a company firewall will not work in this arrangement.

Remember to redeploy after a settings change

As with the other options, `STRAPI_PUBLIC_URL` and `NEXT_PUBLIC_PAYPAL_CLIENT_ID` are frozen into the build. After editing either in Vercel, trigger a fresh deployment — changing the value alone does nothing to the running site.

07 Settings reference

Every setting the two applications read. These are environment variables — entered in a `.env` file, in your hosting panel, or in Vercel's project settings, depending on which option you chose.

Never put these in the code repository
These values are passwords. They are deliberately excluded from the project files. Generate fresh ones for your live site — never reuse the development placeholders, which are published in the project's example files.

CMS — required

Without all six secrets the CMS refuses to start. Generate each one *separately* with `openssl rand -base64 16`.

SETTING	PURPOSE
`APP_KEYS`	Session signing. A comma-separated list of at least two generated values.
`API_TOKEN_SALT`	Protects stored API tokens.
`ADMIN_JWT_SECRET`	Signs admin login sessions.
`TRANSFER_TOKEN_SALT`	Protects data-transfer tokens.
`JWT_SECRET`	Signs API user sessions.
`ENCRYPTION_KEY`	Encrypts sensitive stored values.
`DATABASE_CLIENT`	`postgres`, `mysql` or `sqlite`. Anything else stops the CMS with a clear error.

`DATABASE_HOST`
`DATABASE_PORT`
`DATABASE_NAME`
`DATABASE_USERNAME`
`DATABASE_PASSWORD`
	Your database connection. Not needed for SQLite, which is a single file and is only appropriate for development or a small demo.

`HOST` / `PORT`	Defaults to `0.0.0.0` and `1337`. Leave alone unless your host requires otherwise.

Website — core

SETTING	REQUIRED	PURPOSE
`STRAPI_URL`	Yes	Where the *website's server* fetches content from. On one machine this can be an internal address. Leave it blank and the site runs on its built-in original text.
`STRAPI_PUBLIC_URL`	Yes*	Where a *visitor's browser* loads CMS images from. Must be publicly reachable. Only needed when it differs from `STRAPI_URL` — which it does in Docker and behind any proxy. **Frozen into the build.**
`STRAPI_API_TOKEN`	Yes	Lets the website write to the CMS. **Store orders are not saved without it**, and neither are contact form messages. Created in Section 8.
`REVALIDATE_SECRET`	Optional	A long random password of your choosing that makes CMS edits appear on the site instantly rather than within five minutes. See Section 8.

SETTING	REQUIRED	PURPOSE
`STRAPI_REVALIDATE`	Optional	How many seconds the site caches content. Blank uses the sensible five-minute default. `0` means never cache — useful for a demo, wasteful for a live site. **Affects the build.**
`NODE_ENV`	Yes	Set to `production`.

Website — payments

SETTING	PURPOSE
`NEXT_PUBLIC_PAYPAL_CLIENT_ID`	Your PayPal Client ID. Public by design — it is what draws the PayPal button. **Frozen into the build:** changing it requires a rebuild.
`PAYPAL_CLIENT_SECRET`	Your PayPal secret. Server-side only — never expose it.
`PAYPAL_ENV`	`sandbox` for testing, `live` for real money. Getting this wrong is the classic launch-day mistake.
`STRIPE_SECRET_KEY`	Your Stripe secret key. Setting it enables the "Pay with card" option. Nothing else is needed — Stripe hosts the payment page.
`STRIPE_WEBHOOK_SECRET`	Optional. Only for confirming payments that complete after the customer closes the tab.

Website — forms and email

SETTING	PURPOSE
`RESEND_API_KEY`	Your Resend key. **Without it the contact, booking and prayer forms fail to submit.**
`RESEND_FROM`	The verified sending address, e.g. `Bite Size Theology <hello@yourdomain.com>`.

`CONTACT_TO`
`BOOKING_TO`
`PRAYER_TO`
	Fallback recipients for each form. These are only used when the matching field in the CMS is left blank — the admin's **Settings — Form Emails** screen takes priority, so you can change where messages go without touching the server.

Build-time versus run-time, in one sentence

Most settings are read while the site runs, so changing them just needs a restart — but `STRAPI_PUBLIC_URL`, `NEXT_PUBLIC_PAYPAL_CLIENT_ID` and `STRAPI_REVALIDATE` are baked in when the website is compiled, so changing any of those three requires a full rebuild.

08 First-run setup

Four things to do once, in this order, the first time the system is up.

1 Create your administrator account

Open `https://cms.yourdomain.com/admin`. The first visit asks you to create the account — this is the owner login. Use a strong, unique password. Any default credentials mentioned in the project's development notes apply to a developer's laptop only and do not exist on your server.

2 Import your content

Follow Section 3. Do this before anyone starts editing, because the import replaces whatever is already there.

3 Create the API token

In the admin: **Settings → API Tokens → Create new API Token**. Give it **Full access**. Copy the value *immediately* — it is shown once and never again.

Paste it into the website's `STRAPI_API_TOKEN` setting and restart the website.

Skip this and orders vanish

Customers will be charged correctly — the payment happens with PayPal or Stripe — but the order will never be recorded in your admin, so you will not know what to ship.

4 Make edits appear instantly (optional)

By default the site refreshes its content every five minutes, so an edit can take that long to show. To make it immediate, in the admin go to **Settings → Webhooks → Create new webhook**:

```
URL:     https://yourdomain.com/api/revalidate?secret=YOUR_REVALIDATE_SECRET
Method:  POST
Events:  Entry — create, update, delete
         Media — create, update, delete
```

Use the exact value you set as `REVALIDATE_SECRET` on the website. A mismatch is silently rejected, so test it by editing a headline and reloading.

09 Go-live checklist

Work through these in order. Every one of them catches a real, common failure — do not skip the payment test.

The site itself

- [ ] The main domain loads the homepage over **https://** with a valid certificate.
- [ ] The homepage shows **your** content, not the original placeholder text. If it shows placeholders, the website cannot reach the CMS — check `STRAPI_URL`.
- [ ] The CMS admin loads on its own subdomain, over HTTPS, and you can log in.
- [ ] Edit a headline in the admin, save, reload the site — the change appears.
- [ ] Product and episode images display as real photographs, not grey placeholder blocks. Broken images almost always mean `STRAPI_PUBLIC_URL` is wrong, or was changed without a rebuild.
- [ ] Browse the site on a phone — check the shop, the menu and the checkout.

The shop — test with real money

- [ ] `PAYPAL_ENV` is set to `live` and the keys are your live keys, not sandbox ones.
- [ ] Add an item to the basket, go to checkout, and **complete a real purchase of a cheap item** through PayPal. Refund it afterwards.
- [ ] Repeat with a card, through Stripe.
- [ ] Both orders appear in the admin under **Order**, with the right items, quantities, totals and delivery address.
- [ ] Confirm the money arrived in your PayPal and Stripe dashboards.

The forms

- [ ] Send a test message through the contact form — it arrives in the inbox.
- [ ] Same for the booking form and the prayer request form. They can go to different addresses, configured in the admin under **Settings — Form Emails**.
- [ ] Check the messages are also stored in the admin, so nothing is lost if an email bounces.

Safety

- [ ] The admin password is strong and unique.
- [ ] You have taken your first backup — Section 11 — and you know how to restore it.
- [ ] The API token and all secrets are stored somewhere secure, not in the code repository or a chat message.

10 If something breaks

Every entry below is a problem that has actually happened on this project, with the fix that resolved it.

WHAT YOU SEE	WHAT IT MEANS AND HOW TO FIX IT
The build stops with "Killed", or the server freezes while building	Out of memory. This is the single most common failure. Use a server with 2 GB of RAM, or retry with `NODE_OPTIONS=--max-old-space-size=1536`, or build on a bigger machine and upload the result.
"Cannot find module '@tailwindcss/postcss'"	The website was installed without its development dependencies, which it needs in order to build. Reinstall with `npm ci --include=dev`.
The site loads but shows the original placeholder text	The website cannot reach the CMS, and is falling back to its built-in copy exactly as designed. Check `STRAPI_URL`, and check the CMS is actually running. On some cloud servers a machine cannot reach its own public address — that is what `STRAPI_PUBLIC_URL` exists to solve: point `STRAPI_URL` at the internal address and `STRAPI_PUBLIC_URL` at the public one.
Images are broken or show grey blocks	`STRAPI_PUBLIC_URL` is missing, wrong, or was changed without rebuilding the website. It must be the address a visitor's browser can reach, and it must be `https://` if the site is — a browser blocks insecure images on a secure page.
The CMS will not start: "Missing … secret"	One of the six required secrets from Section 7 is not set.
Database connection error	Check the five `DATABASE_*` values, and confirm the database user is actually attached to the database with permission to use it.
"Unsupported DATABASE_CLIENT"	That setting accepts only `postgres`, `mysql` or `sqlite`. If you chose MySQL, remember its driver must be installed — see Option B.
Checkout buttons are missing	Working as intended: the payment keys are not set. Add them and restart. The PayPal button additionally requires a rebuild, because its Client ID is frozen into the build.
A customer paid but no order appears	`STRAPI_API_TOKEN` is missing or invalid, so the website could not write the order. The payment itself is safe — find it in your PayPal or Stripe dashboard. Fix the token, restart, and test again.
Contact form submissions fail	`RESEND_API_KEY` is not set, or `RESEND_FROM` is not a verified sender on your Resend account.
CMS edits take about five minutes to appear	Normal caching. Set up the webhook in Section 8 to make it instant.
The CMS keeps stopping on its own	Memory limits on shared hosting. Move the CMS to a small virtual server and leave the website where it is.
An application will not start, or returns 502	Wrong start file or folder, or it was never built. Check the application's log first — it will name the problem.

11 Updates & backups

Changing content

Text, images, products, prices, podcast episodes and page settings are all edited in the CMS admin and appear on the site with no deployment and no developer. That is the whole point of the CMS half of this system.

Deploying a code change

OPTION	WHAT TO RUN
A — Docker	Pull the new code, then `docker compose up -d --build`. Your database and uploads are in separate storage and are untouched.
B — hPanel	Pull the code, `npm ci --include=dev`, `npm run build`, re-copy `static` and `public` into the standalone folder, then Restart the app in the panel.
C — Vercel	Push to your main branch. Vercel builds and deploys on its own. The CMS server is updated separately, as in A or B.

Backups

Two things must be backed up, not one

The database holds your text, products and orders. The uploads folder holds your images. A backup of only the database restores a site full of missing pictures.

The CMS's own export command captures both in one archive, which is why it is the recommended method:

```
# Docker:
docker compose exec cms npm run strapi export -- --no-encrypt --file backup-2026-01-31
docker compose cp cms:/app/backup-2026-01-31.tar.gz ./

# Plain server (stop the CMS first if it uses SQLite):
cd cms && npm run strapi export -- --no-encrypt --file backup-2026-01-31
```

- Run this on a schedule — weekly at minimum, daily once the shop is busy.
- Keep copies somewhere other than the server itself.
- Always take one immediately before deploying a code change or a CMS upgrade.
- An archive without `--no-encrypt` is password-protected and useless if you lose the password. Either use the flag and store the file securely, or record the password somewhere you will still have it in a year.
- **Test a restore at least once**, on a spare server. An untested backup is a hope, not a backup.

Ongoing costs to expect

ITEM	TYPICAL
Server (Option A or the CMS half of C)	$10–15/month for 2 GB
Shared hosting (Option B)	Usually already part of your plan
Website hosting on Vercel (Option C)	Free tier is normally sufficient
Domain	$10–20/year
HTTPS certificates	Free (Let's Encrypt)

ITEM	TYPICAL
Email sending (Resend)	Free up to a few thousand messages a month
PayPal / Stripe	No monthly fee — a percentage per sale

Bite Size Theology — deployment guide. Also in the project's `docs/` folder: `DOCKER.md` (Docker details), `STORE-CLIENT-CHECKLIST.md` (accounts and decisions for running the shop). The complete, commented list of every setting lives in `.env.example` and `.env.docker.example` in the project files.